

ARM Assembly Instruction Handbook

Exam-focused Cortex-M3 / Thumb-2 reference derived from the ARM lecture set, all 23 indexed ARM papers, professor assembly templates and historical answer sources.

How to use this during an exam

Start from the operation you need, then check four things before copying a pattern: element width, signedness, flags, and AAPCS preservation. The examples use Keil assembly syntax. Instruction availability and immediate encoding depend on Thumb-2; when an immediate is rejected, load it into a register or use a literal.

AAPCS checkpoint

R0-R3 carry the first four arguments and the return begins in R0. Additional arguments are at the caller's stack. R4-R11 and SP are callee-saved. LR must be saved by any non-leaf routine. Keep SP 8-byte aligned whenever control crosses a public function boundary.

Condition-code selection

Suffix	Meaning	Use
EQ / NE	equal / not equal	zero, sentinel, equality
LO / HS	unsigned < / >=	sizes, indexes, addresses
LS / HI	unsigned <= / >	inclusive unsigned bounds
LT / GE	signed < / >=	signed bytes/words
LE / GT	signed <= / >	signed inclusive bounds
MI / PL	negative / non-negative	sign test
CS / CC	carry set / clear	carry or no-borrow / borrow
VS / VC	overflow set / clear	signed overflow contract

Data movement and constants

MOV, MOVS

Use: Copy a register or encodable immediate. MOVS updates N and Z; use MOV when flags must survive. Large constants may need MOVW/MOVT or LDR =constant.

```
MOV R4, R0; MOVS R6, #0
```

Constraints: MOV cannot encode every 32-bit immediate. Do not use MOVS accidentally between CMP and a conditional branch.

MOVW, MOVT

Use: Build a 32-bit constant without a literal-pool load. MOVW writes the low half; MOVT writes the high half without changing the low half.

```
MOVW R0, #0xCDEF; MOVT R0, #0x89AB
```

Constraints: Use when the assembler supports the Thumb-2 forms. Neither is a memory access.

ADR, ADRL

Use: Form the address of a nearby label using the current PC. ADRL is an assembler expansion for a wider range.

```
ADR R0, table
```

Constraints: Range and alignment are assembler-dependent. LDR R0, =label is more general but uses a literal or relocation.

LDR Rd, =expression

Use: Assembler pseudo-instruction for an address or constant. It becomes MOV/MOVW/MOVT when possible or a PC-relative literal load.

```
LDR R4, =outputData
```

Constraints: Keep LTORG far enough from executed code and within literal-load range. This is not the same as LDR Rd, [Rn].

MRS, MSR

Use: Move from or to special registers such as APSR, CONTROL, MSP, PSP, PRIMASK, CFSR-related system state.

```
MRS R0, APSR; MSR APSR_nzcvq, R1
```

Constraints: Only writable fields may be changed. Privileged registers require privileged execution. Preserve unrelated APSR bits when the paper requires them unchanged.

Loads, stores, arrays and matrices

LDR, STR

Use: Load or store a 32-bit word. Use for `uint32_t/int` arrays and stacked words.

```
LDR R5, [R4, R6, LSL #2]; STR R5, [R0, #8]
```

Constraints: Word index scale is 2. Respect alignment. LDR changes no flags.

LDRB, STRB

Use: Load or store one unsigned byte. LDRB zero-extends into the register.

```
LDRB R3, [R0], #1; STRB R3, [R1], #1
```

Constraints: Use LDRSB, not LDRB, when a signed `int8_t` comparison is required.

LDRSB

Use: Load a signed byte and sign-extend it to 32 bits.

```
LDRSB R7, [R4, R6]
```

Constraints: This is why the insertion-sort answer orders negative bytes correctly.

LDRH, STRH

Use: Load or store a 16-bit halfword. LDRH zero-extends.

```
LDRH R0, [R1, R2, LSL #1]
```

Constraints: Use LDRSH for signed `int16_t` data. Halfword index scale is 1.

LDRD, STRD

Use: Transfer a pair of consecutive words.

```
LDRD R0, R1, [R2]
```

Constraints: Register pairing and address alignment constraints depend on the encoding. Do not assume it updates flags.

Pre/post indexing

Use: Combine a transfer with pointer movement. Pre-index changes the address before access; post-index accesses then changes it.

```
LDRB R3, [R0], #1; STR R2, [R1, #4]!
```

Constraints: Writeback modifies the base register. Avoid unpredictable combinations where the base is also a transferred register.

LDM/STM, LDMIA/STMIA

Use: Transfer multiple registers, useful for blocks and context work.

```
LDMIA R0!, {R2-R5}
```

Constraints: Register order follows register number, not textual order. The base writeback must be intentional.

PUSH, POP

Use: Stack aliases for storing/loading register lists. Save callee-saved registers and LR in non-leaf routines; return with `POP {...,PC}`.

```
PUSH {R4-R7, LR}; ...; POP {R4-R7, PC}
```

Constraints: AAPCS requires SP to be 8-byte aligned at public call boundaries. Pushing an odd count of 32-bit registers misaligns SP unless compensated.

Arithmetic

ADD, ADDS

Use: Add operands. The S form updates N, Z, C and V.

```
ADDS R0, R0, R1
```

Constraints: Use ADDS when the following decision needs flags; use ADD when existing flags must survive.

ADC, ADCS

Use: Add with the current carry flag. Combine with ADDS for multiword addition.

```
ADDS R0, R0, R2; ADCS R1, R1, R3
```

Constraints: Carry-in must be established deliberately. ADC without S preserves flags.

SUB, SUBS

Use: Subtract. SUBS updates flags and is also common for countdown loops.

```
SUBS R2, R2, #1; BNE loop
```

Constraints: After CMP/SUBS, unsigned and signed branches interpret the same flags differently.

SBC, SBCS

Use: Subtract with borrow using inverted carry semantics. Combine with SUBS for multiword subtraction.

```
SUBS R0, R0, R2; SBCS R1, R1, R3
```

Constraints: C=1 means no borrow. This is a frequent source of reversed logic.

RSB, RSBS

Use: Reverse subtraction: operand2 minus operand1. With zero it forms two's-complement negation.

```
RSBS R0, R0, #0
```

Constraints: Negating 0x80000000 overflows in signed arithmetic; inspect V if required.

MUL

Use: Multiply the low 32 bits of two operands.

```
MUL R0, R0, R1
```

Constraints: High overflow is discarded. MUL does not replace UMULL/SMULL when the full 64-bit product is required.

MLA, MLS

Use: Multiply-accumulate or multiply-subtract. MLS is especially useful for remainder after division.

```
UDIV R1, R0, R2; MLS R3, R1, R2, R0
```

Constraints: MLS computes accumulator minus product; verify operand order.

UDIV, SDIV

Use: Unsigned or signed 32-bit division.

```
UDIV R7, R5, R10; SDIV R6, R0, R1
```

Constraints: They return a quotient only and do not set condition flags. Divide-by-zero behavior depends on CCR trapping. Papers may forbid SDIV and require a manual 64/32 algorithm.

UMULL, SMULL, UMLAL, SMLAL

Use: Produce or accumulate a full 64-bit result in a low/high register pair.

```
UMULL R0, R1, R2, R3
```

Constraints: Low and high destinations must be distinct and chosen carefully. Signed and unsigned high halves differ.

Bitwise operations and shifts

AND, ANDS, TST

Use: AND masks bits. ANDS stores the result and sets N/Z; TST sets flags but discards the result.

```
ANDS R0, R0, #0xFF; TST R1, #1
```

Constraints: TST is preferable when only the branch decision matters.

ORR, ORRS

Use: Set or combine bits.

```
ORR R0, R0, R1
```

Constraints: Use BIC to clear known bits rather than ORR with an inverted mask.

EOR, EORS

Use: Exclusive OR, used for parity, toggling, matrix arithmetic and LCG variants.

```
EORS R9, R9, R12
```

Constraints: XORing a bit twice cancels it. The S form overwrites N/Z.

BIC, BICS

Use: Clear selected bits: $Rd = Rn \text{ AND NOT operand2}$.

```
BIC R0, R0, #(3 << 4)
```

Constraints: Useful for read-modify-write register configuration before ORR sets the desired field.

MVN, MVNS

Use: Bitwise NOT of the operand.

```
MVN R0, R0
```

Constraints: Often combined with ADDS #1 for two's complement when RSB is unsuitable.

LSL, LSLs

Use: Logical left shift; fills low bits with zero and can provide an index scale or bit mask.

```
LDR R0, [R4, R6, LSL #2]
```

Constraints: Large shifts and flag behavior differ between immediate and register forms. LSL by 2 scales a word index.

LSR, LSRS

Use: Logical right shift; fills high bits with zero. Use for unsigned fields.

```
LSR R0, R0, #8
```

Constraints: Do not use it to divide a negative signed value; ASR preserves the sign.

ASR, ASRS

Use: Arithmetic right shift; replicates the sign bit.

```
ASR R0, R0, #1
```

Constraints: Rounding for negative values is toward negative infinity, not C signed division's truncation toward zero.

ROR, RRX

Use: Rotate right; RRX rotates through carry by one bit.

```
ROR R0, R0, #8
```

Constraints: RRX consumes and replaces carry, so establish C deliberately.

UXTB, UXTH, SXTB, SXTH

Use: Explicitly zero- or sign-extend an 8- or 16-bit value already in a register.

```
UXTB R5, R1
```

Constraints: Loads may already extend; use these after arithmetic or when documenting a narrow contract.

REV, REV16, REVSH

Use: Reverse byte order in a word, each halfword, or a signed halfword.

```
REV R0, R0
```

Constraints: Use for endianness conversion, not bit reversal.

CLZ

Use: Count leading zero bits.

```
CLZ R0, R1
```

Constraints: Useful for normalization and bit length. Define the desired behavior for input zero.

Comparison and conditional control

CMP, CMN

Use: Set flags for subtraction or addition without storing a result.

```
CMP R6, R5; BHS done
```

Constraints: CMP feeds both signed and unsigned branch families; choose the family that matches the data type.

B

Use: Unconditional branch to a local label.

```
B outer_loop
```

Constraints: It does not save a return address. Use BL for a subroutine call.

BEQ/BNE

Use: Branch when Z is set/not set, typically after CMP, TST or an S-form arithmetic instruction.

```
CMP R0, #0; BEQ empty
```

Constraints: Any intervening flag-setting instruction destroys the comparison result.

BLO/BHS and BCC/BCS

Use: Unsigned lower/higher-or-same; aliases based on carry clear/set.

```
CMP R6, R5; BHS done
```

Constraints: Use for sizes, addresses and uint values. Do not use BLT/BGE for unsigned lengths.

BLS/BHI

Use: Unsigned lower-or-same/higher.

```
CMP R5, #1; BLS done
```

Constraints: These include equality differently from BLO/BHS.

BLT/BGE

Use: Signed less-than/greater-or-equal using N and V.

```
CMP R8, #0; BLT insert
```

Constraints: Required for signed array elements and signed candidates.

BLE/BGT

Use: Signed less-or-equal/greater-than.

```
CMP R0, R7; BLE insert
```

Constraints: Do not substitute unsigned BLS/BHI when negatives are possible.

BMI/BPL

Use: Branch on negative/non-negative according to N.

```
CMP R0, #0; BMI negative
```

Constraints: After arithmetic overflow, N alone may not represent signed relational ordering; CMP plus BLT/BGE is safer.

CBZ/CBNZ

Use: Compare a low register with zero and branch without changing flags.

```
CBZ R0, empty
```

Constraints: Branch range and eligible registers are encoding-dependent. Useful when current flags must survive.

IT/ITE and conditional suffixes

Use: Condition one to four following Thumb instructions. ITE selects then/else forms.

```
CMP R0, #0; ITE EQ; MOVEQ R1,#1; MOVNE R1,#0
```

Constraints: Instructions in the block need matching condition suffixes. Prefer ordinary branches for long or changing logic.

Calls, returns, exceptions and concurrency

BL

Use: Call a subroutine by writing the return address to LR.

```
BL aliquotSum
```

Constraints: A routine that executes BL is non-leaf and must preserve its incoming LR before the first nested call.

BX, BLX

Use: Branch to a register; BX LR is the normal leaf return. BLX also writes LR and may change instruction state.

```
BX LR
```

Constraints: Cortex-M code must remain in Thumb state; function addresses have bit 0 set.

SVC

Use: Enter the supervisor-call exception with an 8-bit immediate service number.

```
SVC #50
```

Constraints: The handler normally decodes the immediate from the halfword at stacked PC minus 2. Select MSP or PSP from EXC_RETURN.

BKPT

Use: Enter the debugger with an immediate breakpoint number.

```
BKPT #0
```

Constraints: Without a debugger, behavior can escalate to a fault. Do not leave test breakpoints in the submitted flow.

CPSID/CPSIE

Use: Disable or enable configurable interrupts, usually with operand i.

```
CPSID i; ...; CPSIE i
```

Constraints: Do not blindly re-enable interrupts if they were already disabled; saving/restoring PRIMASK is safer.

DMB, DSB, ISB

Use: Memory, completion and instruction-stream barriers.

```
DMB; MSR CONTROL, R0; ISB
```

Constraints: Use for synchronization and system-control changes, not as a substitute for volatile or correct ownership.

LDREX/STREX, CLREX

Use: Exclusive load/store pair for lock-free updates. STREX reports whether the reservation succeeded.

```
retry: LDREX R1, [R0]; ADD R1, #1; STREX R2, R1, [R0]; CBZ R2, retry
```

Constraints: Interrupts or other writes may clear the reservation. Always loop on STREX failure.

WFI, WFE, SEV, NOP

Use: Wait for interrupt/event, send event, or execute no operation.

```
WFI
```

Constraints: WFI is safe only when an enabled event can wake the processor. It does not configure the peripheral or clear pending flags.

Assembler directives that appear in the material

AREA

Declare a code or data section and its attributes.

THUMB

Assemble Thumb instructions.

PRESERVE8

State that code preserves 8-byte stack alignment.

EXPORT / IMPORT

Publish or reference linker symbols.

PROC / ENDP

Mark procedure boundaries for the assembler/debugger.

DCD / DCB

Define word or byte constants.

SPACE

Reserve uninitialized bytes; multiply word counts by four.

EQU / RN

Define a constant or register alias.

ALIGN

Align the following location.

LTORG

Emit the current literal pool.

END

End the assembly source.

High-frequency combinations

Bounded byte loop: LDRB/STRB with post-indexing, SUBS counter, BNE. **Signed sort:** LDRSB, CMP, BLE/BLT.

Word array: LDR/STR with index LSL #2. **Non-leaf:** aligned PUSH, BL, matching POP to PC. **Remainder:** UDIV then MLS. **Peripheral field:** LDR register, BIC field mask, ORR desired value, STR back. **Event handoff:** minimal handler, clear W1C flag, update volatile state, foreground processing.